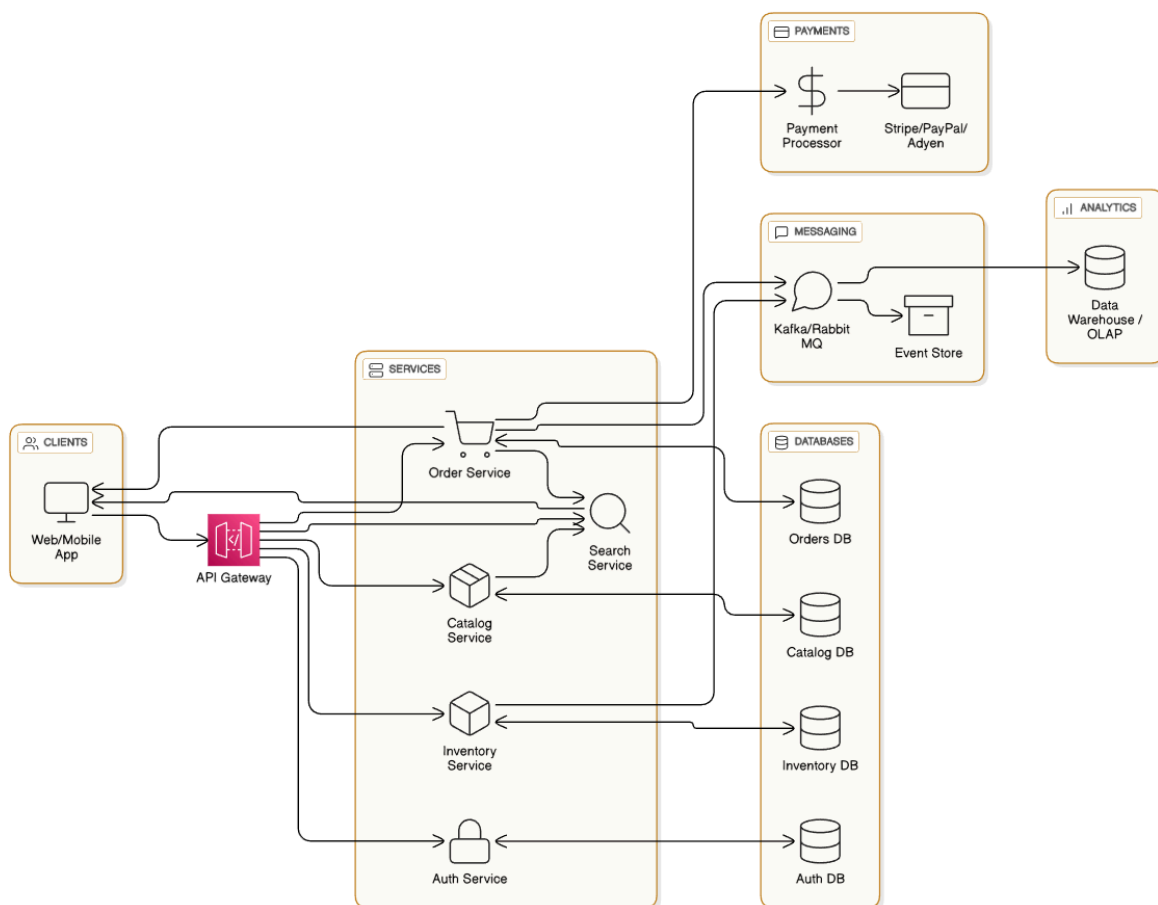


# System Architecture Assessment: Scalable E-commerce Platform Architecture Design

## Requirements

- Handle 10k+ concurrent users
- Support multiple payment gateways
- Real-time inventory management
- Order processing and tracking
- Search and recommendation engine

## High Level Architecture (HLA)



Data flow summary:

1. Clients call API Gateway (auth, rate limits, basic routing).
2. API Gateway routes to microservices (Catalog, Orders, Inventory, Auth, Search).
3. Order placement uses Payment Processor + Order Service. Payment events go through message broker.
4. Inventory is updated via event-driven pipeline (reservation → decrement → commit/rollback).
5. Search engine (Elasticsearch/OpenSearch) is fed from Catalog/Orders for product search & recommendations.
6. Message broker (Kafka/RabbitMQ) decouples services; also used for asynchronous jobs, fan-out notifications, and real-time sync.

## Tech Stack Recommendation

### A. Core infra

- **API Gateway:** Kong / AWS API Gateway / Traefik — central auth, quotas, routing.
- **Container orchestration: Kubernetes** (GKE/EKS/AKS) for autoscaling and deployments. Autoscaling + resource management recommended for 10k+ concurrent users. [ScaleOps](#)

### B. Messaging / streaming

- **Apache Kafka** for high-throughput event streaming (inventory events, order events, analytics). Kafka is better suited for high-throughput, durable streaming compared to traditional brokers. Use Kafka Connect for CDC. [ProjectPro](#)
- RabbitMQ where strict message ordering per queue or simple RPC-style queues are needed (e.g., legacy integrations).

### C. Search & recommendations

- **Elasticsearch** for product search and vector/semantic search (recommendations). Elastic shows strong performance for complex & vector searches; OpenSearch is a strong open-source AWS-integrated alternative — choose based on licensing, feature needs, and vector search performance. [Elastic+1](#)

### D. Datastores

- **Primary OLTP:** PostgreSQL (or cloud-managed equivalent). Strong relational model for orders/payments and transactional guarantees.
- **NoSQL / caching:** Redis for session store, hot caches, counters, and distributed locks (inventory reservation).
- **Data warehouse / analytics:** Snowflake / BigQuery / Redshift for analytics and offline recommendation model training.

### E. Payments

- Integrate multiple gateways: **Midtrans**, **Xendit**, **DOKU**, **Gopay**, or any local providers.

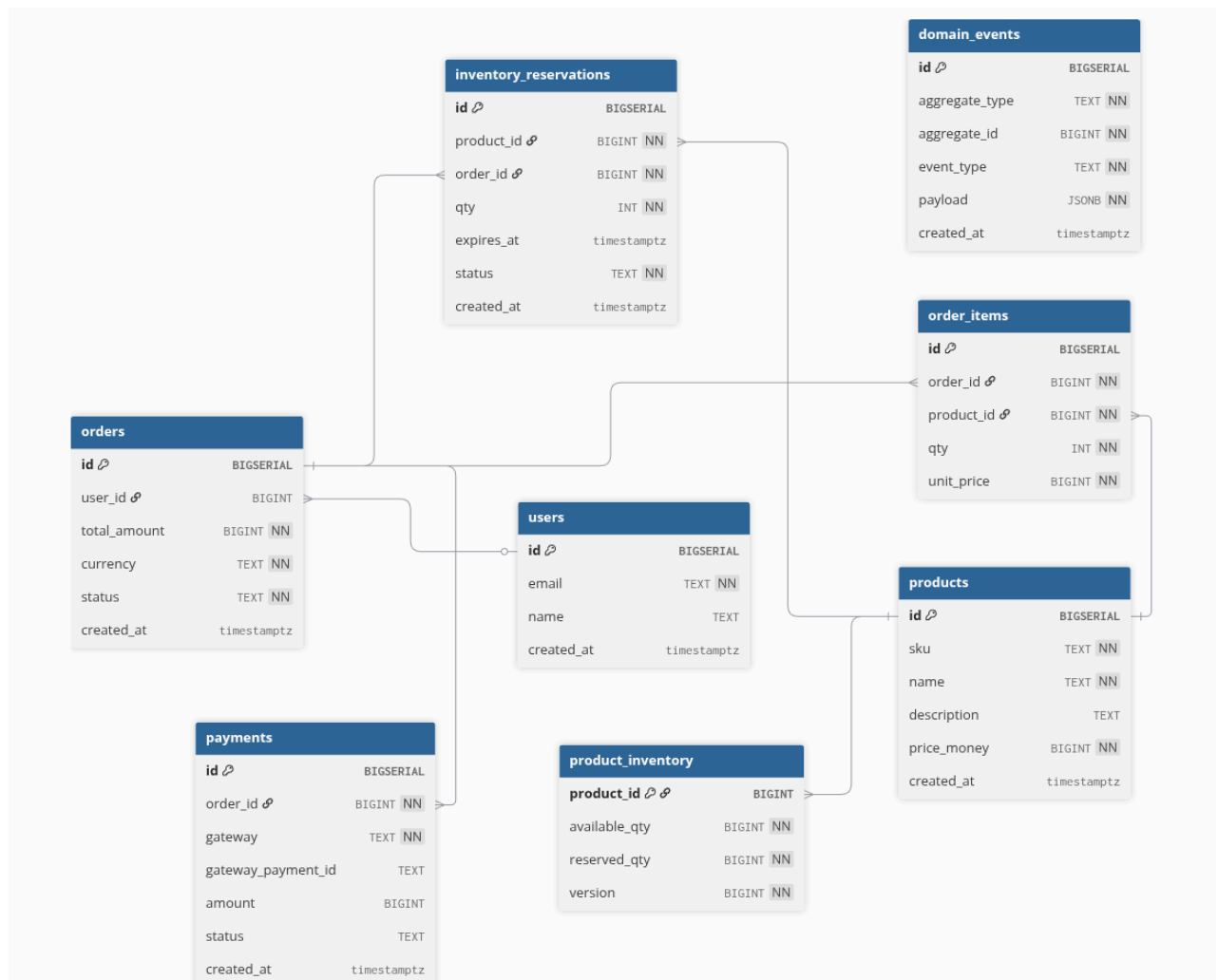
### F. Inventory & events

- Use **Event Sourcing** / event-driven design for inventory and order state transitions (append-only event store + projections). This helps correctness for real-time inventory sync across channels. [Microsoft Learn+1](#)

#### G. Observability

- Prometheus + Grafana for metrics, ELK/Opensearch for logs, Jaeger for tracing.

## Database Schema Design



## API Specification Outline

#### A. Auth / Users

- POST /auth/login → get token
- POST /auth/register
- GET /users/{id}

#### B. Catalog & Search

- GET /products → paginated list (filters)

- GET /products/{id}
  - GET /search?q=... → search service (ES/OpenSearch)
- C. Cart & Orders
- POST /carts/{userId}/items → add to cart
  - POST /orders → create order (reserves inventory)
    - body: cart, payment\_method\_id, shipping\_address
  - GET /orders/{id}
- D. Payments
- POST /payments/{orderId}/initiate → returns gateway redirect / token
  - POST /payments/webhook/{gateway} → webhook endpoint for payment status
- E. Inventory (internal)
- POST /internal/inventory/reserve → reserve items (used by Order svc)
  - POST /internal/inventory/commit → commit reservation (on successful payment)
  - POST /internal/inventory/release → release reservation (on timeout/cancel)
- F. Webhooks & Events
- POST /webhooks/payment (gateway)
  - POST /webhooks/shipping (carrier)
- G. Recommendations & Analytics
- GET /recommendations?userId=...&productId=...

## Scalability and reliability considerations

- A. Handling **10k+ concurrent users**:
- **Kubernetes** autoscaling + cluster autoscaler; set appropriate pod resource requests/limits and horizontal autoscaling based on CPU/RPS/queue length. Use a global load balancer and multiple API Gateway replicas.
  - **Database scaling**: primary for writes (Postgres), read replicas for read-heavy endpoints (catalog, product pages). Use connection pooling (PgBouncer).
  - **Caching**: aggressive caching for product pages (CDN + Redis). Cache search results where appropriate; invalidate on product update.
  - **Message broker** (Kafka) to decouple services and absorb spikes (order events, inventory updates).
  - **Event-driven inventory**: use reservation → commit model.
  - **Sagas** for distributed transactions: implement saga orchestration or choreography for order + payment + inventory (compensating actions on failure).
- B. Fault tolerance & error handling:
- Retry policies with exponential backoff for transient errors (network / gateway).
  - Circuit breaker patterns for third-party calls (payment gateways).
  - Dead-letter queues for failed async messages.
  - Bulkhead isolation: isolate critical services (payments, inventory) into separate resource pools.

C. Data partitioning & sharding

- Partition **orders** / **events** by time or by customer region to make queries and retention manageable.
- For very high scale, shard products by hash to multiple DB clusters.